

Final Project Report

CSE 816: Software Production Engineering Course

**Automated DevOps Pipeline for an MLOps Application
Suite: Doctor Recommendation System**

Master of Technology (M.Tech)
in
Computer Science and Engineering

Submitted by

Meenal Hirwani – MT2025071
Rao Shruti Mohankumar – MT2025101



Submitted to

Department of Computer Science and Engineering
International Institute of Information Technology, Bangalore
Karnataka – 560100, India

May 2026

Abstract

This project presents the design, implementation, and evaluation of a comprehensive MLOps-driven DevOps framework for Doctor Recommendation System developed to automate specialty prediction and doctor recommendation workflows. The primary objective of the project is to deploy and manage machine learning-powered healthcare services through an automated Software Development Life Cycle (SDLC), spanning code integration, containerization, deployment, orchestration, and monitoring using modern DevOps and MLOps practices.

The implemented framework incorporates Git and GitHub for robust version control and collaborative development. Continuous Integration and Continuous Deployment (CI/CD) are automated using Jenkins pipelines defined through a Jenkinsfile, triggered via SCM polling or GitHub integration. The system is designed as a microservices-based architecture comprising three independent services: a Frontend Gateway, a Speciality Predictor service, and a Bandit Recommender service.

The Speciality Predictor microservice utilizes the BioDistillGPT-2 biomedical language model to predict the most relevant medical specialties from patient-described symptoms. The Doctor Recommendation microservice implements an Epsilon-Greedy Multi-Armed Bandit algorithm to rank and recommend doctors based on factors such as ratings and experience while balancing exploration and exploitation strategies.

All services are containerized using Docker to ensure portability, reproducibility, and isolated dependency management. Kubernetes (Minikube) is used as the orchestration platform for automated deployment, service discovery, scalability, and container orchestration and service management. Configuration management and deployment automation are handled using Ansible with templated Kubernetes manifests. Centralized logging and monitoring are implemented using Elasticsearch and Kibana, enabling operational visibility into system performance and application behavior.

The project successfully demonstrates a complete end-to-end MLOps workflow where code changes pushed to GitHub automatically trigger Jenkins pipelines for Docker image creation, image deployment, Kubernetes rollout, and application updates. The deployed system provides a scalable, maintainable, and prototype doctor recommendation platform capable of delivering intelligent doctor recommendations through automated ML operations.

This work highlights the importance of integrating DevOps and MLOps practices to streamline deployment, improve reliability, enhance scalability, and accelerate the operationalization of machine learning applications in real-world healthcare systems. The complete source code, configuration files, and implemented DevOps pipeline for this project are publicly available on GitHub :

GitHub Repository:

<https://github.com/shrutirao1210/MLOps-Doctor-Recommendation-System>

Acknowledgements

We would like to express our sincere gratitude to our course instructor, Prof. B. Thangaraju, for invaluable guidance, insightful feedback, and continuous support throughout the duration of this project.

We are also thankful to the Department of Computer Science and Engineering and the International Institute of Information Technology, Bangalore for providing the necessary resources and academic environment that made this project possible.

Finally, we would like to thank our families, friends, and peers for their encouragement and support during this work.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1 The Rise of MLOps: Bridging Development and Operations	1
2 Problem Statement:Challenges in MLOps Lifecycle Management	1
3 Project Aim and Objectives: Automating the MLOps Workflow	2
3.1 Aim	2
3.2 Objectives	2
4 Scope of the Project	3
4.1 In Scope	3
4.2 Out of Scope	4
5 Report Structure	4
2 Application Suite Overview:Doctor Recommendation System	6
1 Frontend Gateway Service	6
1.1 Functional Description	6
1.2 Core Components	6
2 Speciality Predictor Service	7
2.1 Functional Description	7
2.2 Core Components	7
3 Bandit Recommender Service	7
3.1 Functional Description	7
3.2 Core Components	7
4 Rationale for Application Suite Selection	8
5 Interaction Between Services	9
3 DevOps Toolchain and System Architecture	10
1 Chosen DevOps Philosophy and Principles	10
2 Selected DevOps Tools and Technologies	11
2.1 Version Control: Git & GitHub	11
2.2 CI/CD Automation: Jenkins	11
2.3 Containerization: Docker and Docker compose	11
2.4 Orchestration Platform: Kubernetes (k8s)	12
2.5 Configuration Management: Ansible	12
2.6 Monitoring and Visualization: Elasticsearch and Kibana	12
3 Overall DevOps System Architecture Diagram	12
4 Data Flow within the DevOps Pipeline	14

4	Version Control and CI/CD Automation with Jenkins	18
1	Git Repository Structure and Management	18
1.1	Repository Organisation	18
1.2	Branching Strategy	19
2	Jenkins Setup and Configuration	19
2.1	Jenkins Architecture	19
2.2	Key Jenkins Plugins Utilised	20
2.3	GitHub Integration: SCM Polling	20
3	Jenkins Pipeline Implementation (Jenkinsfile)	20
3.1	Pipeline-as-Code with Jenkinsfile	20
3.2	Stage 1: Checkout Code	21
3.3	Stage 2: Build Docker Images	21
3.4	Stage 3: Push Docker Images to Docker Hub	21
3.5	Stage 4: Load Images into Minikube	22
3.6	Stage 5: Deploy with Ansible	22
4	Automated Triggers and Notifications	23
5	Containerization with Docker and Docker Compose	24
1	Application Architecture and Containerization Strategy	24
2	Dockerfile for the Bandit Recommendation Service	25
2.1	Dockerfile Structure	25
2.2	Explanation of Key Instructions	25
3	Dockerfile for the Speciality Predictor Service	26
3.1	Dockerfile Structure	26
3.2	Explanation of Key Instructions	27
4	Dockerfile for the Frontend Gateway	27
4.1	Dockerfile Structure	28
4.2	Explanation of Key Instructions	28
5	Building Docker Images	28
6	Pushing Docker Images to Docker Hub	29
7	Loading Images into Minikube	29
8	Multi-Container Service Communication	29
6	Configuration Management with Ansible	31
1	Role of Ansible in This Project	31
2	Ansible Inventory Management	32
3	Ansible Playbook: deploy.yml	32
4	Rendering Kubernetes Manifests from Jinja2 Templates	33
5	Applying Kubernetes Deployments	34
6	Deployment Rollout Verification	34
7	Frontend Service Environment Variables	35
8	Use of Variables and Jinja2 Templates	35
9	Idempotency Considerations	36
10	Relationship Between Jenkins and Ansible	36
11	Summary	36
7	Orchestration and Deployment with Kubernetes	37
1	Kubernetes Cluster Overview	37
2	Application Architecture in Kubernetes	38

3	Bandit Recommendation Kubernetes Deployment	38
3.1	Bandit Deployment Configuration	38
3.2	Bandit Readiness and Liveness Probes	39
3.3	Bandit Service Configuration	39
4	Speciality Predictor Kubernetes Deployment	39
4.1	Speciality Deployment Configuration	40
4.2	TCP Health Checks	40
4.3	Speciality Service Configuration	40
5	Frontend Gateway Kubernetes Deployment	41
5.1	Frontend Deployment Configuration	41
5.2	Frontend Health Checks	41
5.3	Frontend Service Exposure	41
6	Service Discovery and Inter-Service Communication	42
7	Rolling Update Deployment Strategy	42
8	Health Check Design Summary	42
9	Summary	43
8	Monitoring and Logging with Elasticsearch and Kibana	44
1	Role of Monitoring in the Doctor Recommendation System	44
2	Logging Namespace Architecture	45
3	Elasticsearch Deployment	45
3.1	Elasticsearch Deployment Configuration	45
3.2	Memory Configuration	46
3.3	Security Configuration	46
3.4	Elasticsearch Service Configuration	46
4	Kibana Deployment	46
4.1	Kibana Deployment Configuration	47
4.2	Connecting Kibana to Elasticsearch	47
4.3	Kibana Service Exposure	47
5	Centralized Logging Workflow	48
6	Application Monitoring Use Cases	48
6.1	Deployment Monitoring	48
6.2	Inter-Service Communication Monitoring	49
6.3	Container Health Monitoring	49
7	Namespace Separation and Infrastructure Isolation	49
8	Limitations and Future Improvements	50
9	Summary	50
9	System Demonstration, Testing, and Evaluation	51
1	Overview of System Execution	51
2	Successful Jenkins Pipeline Execution	51
3	Verifying Docker Image Availability on Docker Hub	52
4	Frontend User Interface	53
5	Integrated Specialty Prediction and Doctor Recommendation	54
6	Elastic Stack Monitoring Dashboard	55
7	Elastic Observability Integration	56
8	System Validation	57
10	Conclusion and Future Scope	58

1	Conclusion	58
2	Future Scope	59

Chapter 1

Introduction

1 The Rise of MLOps: Bridging Development and Operations

In recent years, Machine Learning (ML) has transitioned from a primarily research-oriented field to a cornerstone of innovation across diverse industries. Applications powered by ML, such as image recognition, natural language processing, and predictive analytics, are increasingly integrated into software products and services, delivering significant value. However, the journey from developing an ML model to successfully deploying and maintaining it in a production environment presents unique and complex challenges.

This gap between ML model development (Dev) and IT operations (Ops) has led to the emergence of MLOps (Machine Learning Operations). MLOps is a set of practices that aims to deploy and maintain ML models in production reliably and efficiently. It extends DevOps principles to the ML lifecycle, emphasizing automation, collaboration, continuous integration, continuous delivery, and continuous monitoring to streamline the entire process from data ingestion and model training to deployment and operational oversight. The goal of MLOps is to increase automation and improve the quality of production ML while also focusing on business and regulatory requirements.

2 Problem Statement: Challenges in MLOps Lifecycle Management

While the potential of intelligent doctor recommendation systems is significant, their operationalization through manual or ad-hoc deployment processes introduces several challenges related to scalability, reproducibility, monitoring, and maintainability. These challenges include:

- **Manual and Error-Prone Deployments:** Without automation, deploying ML models and their associated application components is often a manual, time-consuming, and error-prone process, leading to inconsistencies across environments and potential downtime.
- **Lack of Reproducibility:** Ensuring that ML experiments, model builds, and deployments are reproducible is critical. Manual steps and unmanaged dependencies make reproducibility difficult to achieve.

- **Scalability and Performance Bottlenecks:** ML applications, especially those dealing with tasks like image processing, can be resource-intensive. Manually scaling these applications to meet fluctuating demands is inefficient and often leads to either over-provisioning or performance degradation.
- **Monitoring and Observability Gaps:** Traditional monitoring solutions may not be sufficient for ML systems. Understanding model performance, data drift, and system health requires specialized monitoring and logging, which is often an afterthought in manual setups.
- **Slow Iteration Cycles:** The inability to quickly iterate, test, and deploy new model versions or application updates hampers innovation and responsiveness to changing requirements.
- **Configuration Drift:** Inconsistent configurations across development, testing, and production environments can lead to unexpected behavior and failures.

Addressing these challenges is crucial for realizing the full potential of ML applications in a production setting. This project aims to tackle these issues by implementing a robust DevOps framework.

3 Project Aim and Objectives: Automating the MLOps Workflow

3.1 Aim

The primary aim of this project is to design, implement, and evaluate a comprehensive and automated DevOps framework tailored to streamline the entire lifecycle of the Doctor Recommendation platform. The system specifically comprises a Frontend Gateway, a Speciality Predictor microservice powered by BioDistillGPT-2, and a Bandit-based Doctor Recommendation module. The project demonstrates how DevOps and MLOps practices can improve the deployment, scalability, monitoring, and production readiness of intelligent healthcare recommendation systems.

3.2 Objectives

To achieve the stated aim, the following specific objectives were established:

1. To develop a modular microservices-based healthcare Doctor Recommendation System consisting of a Frontend Gateway, Speciality Predictor service, and Bandit Recommender service for automated doctor recommendation workflows.
2. To implement a biomedical NLP-based speciality prediction module using BioDistillGPT-2 capable of analysing patient symptom descriptions and predicting the most relevant medical specialities.
3. To design and implement an Epsilon-Greedy Multi-Armed Bandit recommendation engine capable of balancing exploration and exploitation while ranking doctors based on ratings and experience.

4. To containerize all application components using Docker, ensuring portability, consistency, reproducibility, and isolated dependency management across environments.
5. To orchestrate and manage deployment of the microservices architecture using Kubernetes (Minikube), enabling scalable and fault-tolerant application execution.
6. To establish an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins and GitHub for automated build, testing, and deployment workflows.
7. To automate infrastructure configuration and deployment management using Ansible playbooks for reproducible environment provisioning and application setup.
8. To implement centralized logging and monitoring using the Elasticsearch and Kibana for operational visibility, debugging, and performance monitoring.
9. To demonstrate a complete end-to-end MLOps workflow where code updates trigger automated CI/CD pipelines and seamless deployment of updated healthcare recommendation services.

4 Scope of the Project

The scope of this Software Production Engineering project is defined as follows:

4.1 In Scope

- Development and implementation of a complete MLOps-driven healthcare recommendation system using a microservices architecture. The system includes a Frontend Gateway, a Speciality Predictor service, and a Bandit Recommender service.
- Implementation of a biomedical Natural Language Processing (NLP) pipeline using BioDistillGPT-2 to analyse patient symptom descriptions and predict relevant medical specialities.
- Application of reinforcement learning principles through an Epsilon-Greedy Multi-Armed Bandit algorithm for intelligent doctor recommendation and ranking based on doctor ratings and experience.
- Full integration and utilization of the following DevOps and MLOps tools as part of the project workflow:
 - Version Control: Git and GitHub.
 - CI/CD Automation: Jenkins pipelines with automated build and deployment workflows.
 - Containerization: Docker with separate Dockerfiles for each microservice and Docker Compose for local multi-container execution.
 - Configuration Management: Ansible playbooks for automated deployment and environment setup.
 - Orchestration and Scaling: Kubernetes (Minikube) with deployment and service manifests for container orchestration.

- Monitoring and Logging: Elasticsearch and Kibana for centralized monitoring, visualization, and operational logging.
- Demonstration of automated software delivery workflows including container image creation, CI/CD-triggered deployments, Kubernetes-based orchestration, and centralized observability.
- Deployment and testing of all microservices in a containerized environment to ensure scalability, portability, and reproducibility.

4.2 Out of Scope

- Real-time integration with hospital management systems, Electronic Health Records (EHR), or live healthcare databases is outside the scope of this project.
- The system is designed as a recommendation and assistance platform only and does not provide certified medical diagnosis or treatment decisions.
- Development of new biomedical language models or advanced reinforcement learning algorithms from scratch is not included. The project utilizes pre-trained BioDistillGPT-2 and a standard Epsilon-Greedy Multi-Armed Bandit approach.
- Advanced security mechanisms such as end-to-end encryption, multi-factor authentication, and healthcare compliance standards (HIPAA/GDPR) are not fully implemented in this prototype.
- Deployment to large-scale cloud-managed Kubernetes platforms such as AWS EKS, Google GKE, or Azure AKS is beyond the scope of this implementation. The project primarily targets local Kubernetes environments using Minikube.
- Continuous model retraining, automated data collection from real users, and production-scale feedback learning pipelines are not implemented in the current system.
- High-availability distributed infrastructure, multi-region deployment, and enterprise-grade fault tolerance mechanisms are outside the present project scope.

5 Report Structure

This report is organized into the following chapters to detail the design, implementation, and evaluation of the automated MLOps-based healthcare recommendation framework:

- **Chapter 2: Application Suite Overview:** Describes the overall Doctor Recommendation system architecture, microservices workflow, and interactions between the Frontend Gateway, Speciality Predictor, and Bandit Recommender services.
- **Chapter 3: DevOps Toolchain and System Architecture:** Explains the selected DevOps and MLOps tools, their roles, and the overall deployment architecture of the integrated healthcare recommendation platform.
- **Chapter 4: Version Control and CI/CD Automation with Jenkins:** Details the GitHub repository structure, Jenkins pipeline design, automated build workflows, and CI/CD integration process.

- **Chapter 5: Containerization with Docker and Docker Compose:** Discusses Docker-based containerization of the microservices, image creation, dependency isolation, and local multi-container execution using Docker Compose.
- **Chapter 6: Configuration Management with Ansible:** Describes the use of Ansible playbooks for automating deployment setup, application configuration, and infrastructure provisioning.
- **Chapter 7: Orchestration and Deployment with Kubernetes:** Explains Kubernetes deployment strategies, service manifests, pod orchestration, scaling concepts, and Minikube-based deployment.
- **Chapter 8: Logging and Monitoring using Elasticsearch and Kibana:** Presents the implementation of centralized logging, monitoring dashboards, and operational visibility using Elasticsearch and Kibana.
- **Chapter 9: System Demonstration, Testing, and Evaluation:** Demonstrates the end-to-end execution of the healthcare recommendation system, testing workflows, and evaluation of deployment and recommendation performance.
- **Chapter 10: Challenges, Conclusion, and Future Work:** Summarizes the implementation challenges, key learnings, conclusions, and possible future enhancements to the project.

Appendices provide supplementary materials including configuration files, Kubernetes manifests, Jenkins pipeline scripts, Ansible playbooks, and additional implementation details.

Chapter 2

Application Suite Overview: Doctor Recommendation System

This chapter provides an overview of the Doctor Recommendation System that forms the core workload for the MLOps framework implemented in this project. The application suite consists of three interconnected microservices: a Frontend Gateway service, a Speciality Predictor service, and a Bandit Recommender service. Understanding the functionalities, architecture, and interactions of these components is essential for appreciating the DevOps and MLOps processes designed to automate, deploy, monitor, and manage their lifecycle.

1 Frontend Gateway Service

The Frontend Gateway acts as the primary user interaction layer of the Doctor Recommendation system. It provides a web-based interface through which users can enter symptoms and receive doctor speciality recommendations. The frontend service communicates with backend microservices using REST APIs and displays prediction results in a user-friendly format.

1.1 Functional Description

The main functionality of the Frontend Gateway is to collect patient symptoms from the user and forward the request to the backend prediction services. Based on the responses returned from the backend services, the frontend dynamically displays the predicted medical speciality recommendations along with ranking information. The service improves accessibility and usability by providing an interactive healthcare recommendation interface.

1.2 Core Components

The Frontend Gateway contains the following major components:

- **Frontend Web Interface:** Provides the user interface for symptom input and recommendation display using FastAPI routes, HTML templates, and static web pages.
- **API Integration Layer:** Handles communication with the backend FastAPI microservices through HTTP requests.
- **Styling Components:** Implements responsive UI design using HTML, CSS, and JavaScript frameworks.

2 Speciality Predictor Service

The Speciality Predictor service is responsible for predicting the most relevant medical speciality based on user-provided symptoms. This microservice uses a BioDistillGPT-2 biomedical language model for biomedical text classification.

2.1 Functional Description

The service receives symptom descriptions from the frontend application and processes the textual input using Natural Language Processing (NLP) techniques. The transformer model analyzes the symptoms and predicts the most appropriate medical speciality category. The predicted results are then forwarded to the recommendation service for ranking and filtering.

2.2 Core Components

The Speciality Predictor service includes the following components:

- **FastAPI Backend Server:** Exposes REST API endpoints for prediction requests.
- **BioDistillGPT-2 Model:** A transformer-based biomedical language model for speciality classification tasks.
- **Tokenizer and Preprocessing Pipeline:** Converts textual symptom descriptions into processed input suitable for model prediction.
- **Prediction Engine:** Performs model inference and returns speciality prediction probabilities.
- **Dependencies:** Includes libraries such as PyTorch, Transformers, FastAPI, NumPy, and Pandas.

3 Bandit Recommender Service

The Bandit Recommender service is designed to rank and recommend doctor specialities using reinforcement learning principles based on user interaction and feedback.

3.1 Functional Description

This service receives predicted specialities from the predictor service and applies an Epsilon-Greedy Multi-Armed Bandit algorithm to rank recommendations dynamically. The recommender balances exploration and exploitation strategies during recommendation generation.

3.2 Core Components

The Bandit Recommender service consists of the following components:

- **Recommendation Engine:** Implements the Epsilon-Greedy Multi-Armed Bandit algorithm.
- **FastAPI Service Layer:** Provides API endpoints for recommendation requests.

- **Weight Adjustment Mechanism:** Allows dynamic adjustment of rating and experience weights during recommendation generation.
- **Recommendation Ranking Logic:** Generates prioritized speciality recommendations.
- **Dependencies:** Includes FastAPI, NumPy, Pandas, and Python-based reinforcement learning utilities.

4 Rationale for Application Suite Selection

The Doctor Recommendation System was selected as the target application suite for this project because it represents a practical and realistic MLOps workload involving multiple Machine Learning services, microservice communication, and real-time prediction pipelines.

The rationale behind selecting this application suite includes the following factors:

- **Healthcare Domain Relevance:** The project addresses a real-world healthcare problem by assisting users in identifying relevant medical specialties based on symptom descriptions. Healthcare applications require reliable deployment, monitoring, scalability, and maintainability, making them highly suitable for MLOps implementation.
- **Integration of NLP and Recommendation Systems:** The application combines Natural Language Processing using a BioDistillGPT-2 model with a reinforcement learning-based recommendation engine. This demonstrates how different ML paradigms can co-exist within a unified production system.
- **Microservices-Based Architecture:** The system consists of multiple independently deployable services including the Frontend Gateway, Speciality Predictor service, and Bandit Recommender service. Such an architecture provides an ideal environment for demonstrating containerization, orchestration, CI/CD automation, and service communication.
- **Complex Artifact Management:** The project involves management of multiple artifacts such as trained transformer models, tokenizer files, Docker images, Kubernetes manifests, Jenkins pipelines, and configuration files. Handling these artifacts efficiently is a key challenge in MLOps workflows.
- **Scalability and Deployment Challenges:** Machine Learning inference services often require scalable infrastructure due to varying workloads and computational demands. The project demonstrates how Kubernetes and Docker can be used to manage scalability and deployment automation effectively.
- **Real-Time Prediction Workflow:** The system performs real-time symptom analysis and recommendation generation through API-based communication between services. This makes the project suitable for demonstrating automated deployment and monitoring pipelines.
- **Practical Demonstration of MLOps Concepts:** The application provides sufficient complexity to showcase important MLOps practices such as CI/CD automation, containerization, orchestration, centralized logging, monitoring, and infrastructure management.

5 Interaction Between Services

The Doctor Recommendation System follows a microservices-based architecture where all services communicate using REST APIs.

The interaction workflow is as follows:

1. The user enters symptoms through the Frontend Gateway interface.
2. The frontend sends the symptom data to the Speciality Predictor service.
3. The Speciality Predictor processes the symptoms using the BioDistillGPT-2 model and predicts relevant medical specialities.
4. The prediction results are forwarded to the Bandit Recommender service.
5. The recommender service ranks the recommendations using the Epsilon-Greedy algorithm.
6. The ranked recommendations are returned to the frontend and displayed to the user.

This modular architecture enables independent deployment, scaling, monitoring, and maintenance of each service within the MLOps pipeline.

Chapter 3

DevOps Toolchain and System Architecture

This chapter describes the DevOps tools, automation workflow, and overall system architecture implemented for the Doctor Recommendation System. The project integrates multiple microservices, containerization, CI/CD automation, orchestration, and monitoring components to ensure scalable and reliable deployment of the healthcare recommendation platform.

1 Chosen DevOps Philosophy and Principles

The design and implementation of the DevOps framework for this doctor recommendation platform are guided by principles focused on automation, scalability, reproducibility, and efficient deployment management.

- **Microservices-Based Architecture:** The project follows a microservices-oriented design where the Frontend Gateway, Speciality Prediction Service, and Bandit Recommendation Service operate as independent deployable components. This architecture improves modularity, maintainability, and scalability.
- **Continuous Integration (CI):** GitHub and Jenkins are used to support continuous integration workflows. Source code updates trigger automated build and deployment processes, helping maintain deployment consistency and reducing manual intervention.
- **Continuous Deployment (CD):** The deployment workflow automates container image deployment and Kubernetes rollout processes. This enables faster and more reliable application updates across services.
- **Containerization with Docker:** All application services are containerized using Docker to ensure dependency isolation, portability, and reproducible execution environments across systems.
- **Infrastructure as Code (IaC):** Infrastructure configuration and deployment automation are managed using Kubernetes manifests and Ansible playbooks. This approach improves consistency and simplifies operational management.
- **Scalability and Orchestration:** Kubernetes is used to orchestrate deployment and management of the containerized services. The orchestration layer supports service scalability, availability, and simplified deployment handling.

- **Monitoring and Observability:** The project integrates Elasticsearch and Kibana for centralized monitoring and log visualization. These tools improve system observability, debugging capability, and operational monitoring.
- **Automation-Driven Deployment:** The implemented DevOps workflow minimizes manual deployment effort through automated pipelines, infrastructure configuration, and service orchestration processes.

2 Selected DevOps Tools and Technologies

A collection of open-source DevOps tools and technologies has been integrated to automate deployment, orchestration, monitoring, and lifecycle management of the doctor recommendation platform. The role of each selected tool is summarized below.

2.1 Version Control: Git & GitHub

- **Tool:** Git and GitHub.
- **Rationale:** Git is used for distributed version control and source code management across all application services and deployment configurations. GitHub serves as the centralized repository hosting platform for maintaining application source code, Kubernetes manifests, Docker configurations, Jenkins pipeline files, and Ansible playbooks.

Version control enables collaborative development, change tracking, rollback capability, and seamless integration with automated deployment workflows.

2.2 CI/CD Automation: Jenkins

- **Tool:** Jenkins automation server.
- **Rationale:** Jenkins is used to automate Continuous Integration and Continuous Deployment workflows for the healthcare recommendation platform.

The implemented Jenkins pipeline automates source code integration, Docker image generation, deployment execution, and Kubernetes rollout processes. The use of pipeline automation reduces manual operational effort and improves deployment consistency and reproducibility.

2.3 Containerization: Docker and Docker compose

- **Tool:** Docker and Docker Compose.
- **Rationale:** Docker is used to containerize the frontend gateway, speciality prediction microservice, and bandit recommendation microservice along with their dependencies.

Containerization ensures consistent runtime environments, portability, dependency isolation, and reproducible deployments across systems. Docker Compose is additionally used for simplified local multi-container execution and testing.

2.4 Orchestration Platform: Kubernetes (k8s)

- **Tool:** Kubernetes.
- **Rationale:** Kubernetes is used for orchestrating deployment and management of the containerized services.

The orchestration platform manages pod deployment, service exposure, replica handling, and application scalability. Kubernetes improves service availability, deployment automation, and operational management of the distributed healthcare recommendation platform.

2.5 Configuration Management: Ansible

- **Tool:** Ansible.
- **Rationale:** Ansible is used for automating deployment configuration and infrastructure management operations.

The implemented playbooks automate deployment tasks, service configuration, Kubernetes deployment triggering, and environment setup activities. Infrastructure automation improves deployment consistency and minimizes manual administrative operations.

2.6 Monitoring and Visualization: Elasticsearch and Kibana

- **Tool:** Elasticsearch and Kibana.
- **Rationale:** Elasticsearch and Kibana are integrated for centralized monitoring and visualization of system activities and logs.

These tools provide operational visibility, log analysis, debugging support, and application monitoring capabilities for the deployed healthcare recommendation services.

3 Overall DevOps System Architecture Diagram

The integrated DevOps toolchain forms a cohesive system architecture designed to automate the MLOps lifecycle. Figure 3.1 illustrates the high-level architecture and the flow of operations.

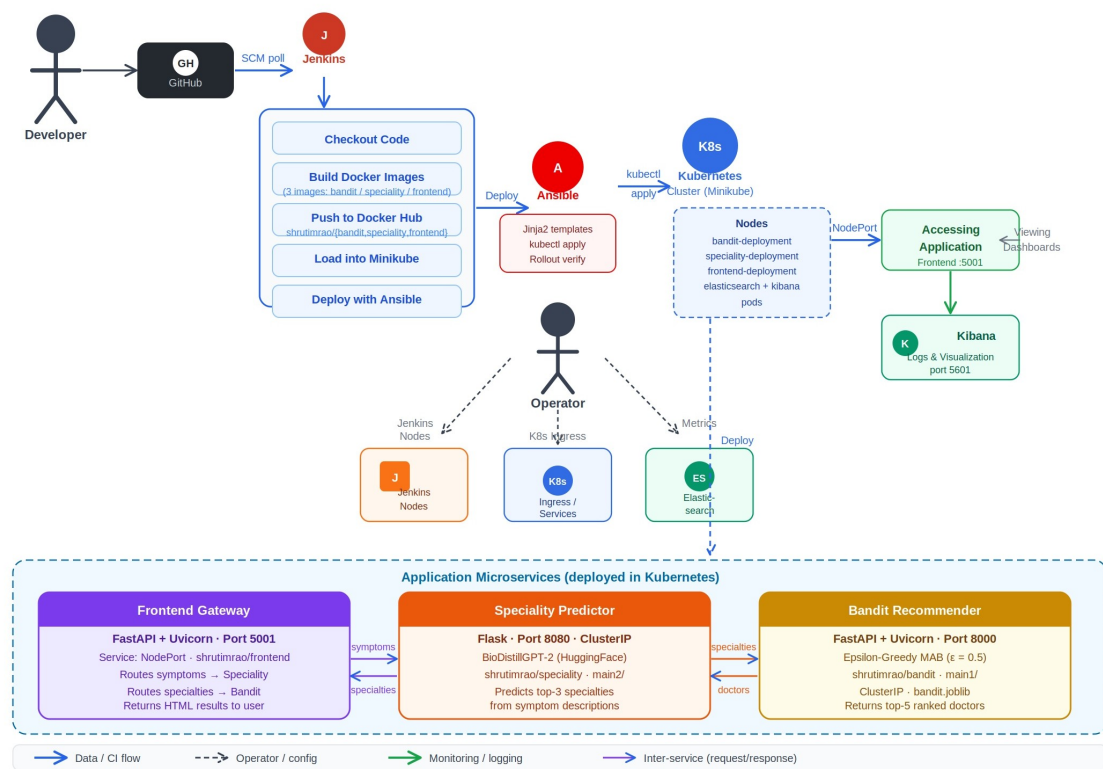


Figure 3.1: Overall DevOps System Architecture for the Doctor Recommendation System

4 Data Flow within the DevOps Pipeline

Understanding the flow of data, configuration files, deployment artifacts, and runtime communication within the DevOps pipeline is essential for understanding the operational workflow of the Doctor Recommendation MLOps system. The implemented pipeline ensures that code changes, Docker images, Kubernetes deployments, and application requests move through a structured and automated lifecycle.

1. Source Code and Configuration Flow

The development workflow begins when developers update and commit application source code, configuration files, Kubernetes manifests, Dockerfiles, Jenkins pipeline definitions, and Ansible playbooks to the GitHub repository.

The repository contains the complete source code for all three microservices of the Doctor Recommendation platform, including:

- Frontend Gateway service
- Speciality Predictor microservice
- Bandit Recommender microservice

In addition to application code, the repository also stores:

- Dockerfiles for containerization
- Jenkinsfile for CI/CD automation
- Ansible deployment playbooks
- Kubernetes deployment and service manifests

Jenkins continuously monitors the GitHub repository using SCM polling. Whenever new commits are detected in the main branch, Jenkins automatically fetches the latest repository state and initiates the CI/CD pipeline execution process.

2. Build Artifact Flow

After the source code is retrieved from GitHub, Jenkins proceeds to build the application artifacts required for deployment.

The pipeline creates separate Docker images for the three application services:

- Frontend Gateway
- Speciality Predictor
- Bandit Recommender

Each image is generated using its corresponding Dockerfile. During the build process:

- Python dependencies are installed
- Application files are copied into the container
- Runtime environments are configured

- Exposed ports are defined

The Docker images are tagged using the `:latest` tag and prepared for deployment. Jenkins also stores build logs and console outputs for debugging, auditing, and pipeline monitoring purposes.

3. Container Image Flow

Once the Docker images are successfully built, Jenkins pushes the images to Docker Hub using securely stored Docker Hub credentials configured inside Jenkins.

The following application images are pushed:

- `shrutimrao/frontend`
- `shrutimrao/speciality`
- `shrutimrao/bandit`

After pushing the images to Docker Hub, Jenkins loads the same images into the Minikube image cache using the `minikube image load` command.

This approach provides several advantages:

- Faster Kubernetes deployment
- Reduced external image pull dependency
- Improved deployment reliability
- Lower deployment latency

Kubernetes deployments use the locally cached images through the `imagePullPolicy: IfNotPresent` configuration.

4. Deployment Configuration Flow

The deployment stage is managed using Ansible automation integrated within the Jenkins pipeline.

Ansible dynamically renders Kubernetes deployment manifests from Jinja2 template files. These templates contain deployment parameters such as:

- Docker image names
- Image tags
- Replica counts
- Container ports
- Service configurations

The rendered YAML manifests are generated temporarily and applied to the Kubernetes cluster using the `kubectl apply` command.

The deployment process creates or updates:

- Kubernetes Deployments

- Kubernetes Services
- Internal networking configurations

Kubernetes then reconciles the desired cluster state with the current state and performs rolling updates whenever newer container versions are deployed.

Ansible additionally verifies deployment success using rollout status checks to ensure that all application pods become operational successfully before the pipeline completes execution.

5. Runtime Data and Request Flow

Once deployment is completed, the Doctor Recommendation System application becomes accessible through the Frontend Gateway service exposed using a Kubernetes NodePort service.

The runtime request flow proceeds as follows:

- (a) Users enter symptom descriptions through the frontend web interface.
- (b) The Frontend Gateway forwards the symptom text to the Speciality Predictor microservice using Kubernetes internal service communication.
- (c) The Speciality Predictor processes the text using the BioDistillGPT-2 language model and predicts the most relevant medical specialties.
- (d) The predicted specialties are returned to the Frontend Gateway.
- (e) The Frontend Gateway forwards the specialties to the Bandit Recommender service.
- (f) The Bandit Recommender applies the epsilon-greedy Multi-Armed Bandit algorithm to rank doctors based on ratings, experience, and specialty relevance.
- (g) The ranked doctor recommendations are returned to the frontend and displayed to the user.

Kubernetes internal DNS enables communication between services using service names, allowing secure and scalable inter-service networking.

6. Monitoring and Log Flow

All deployed microservices generate runtime logs through standard output and standard error streams.

These logs are collected from Kubernetes pods and made available for centralized monitoring using the ELK Stack components deployed within the Kubernetes cluster.

The monitoring workflow includes:

- Elasticsearch for log storage and indexing
- Kibana for visualization and dashboard monitoring

Kibana dashboards provide operational visibility into:

- Application execution logs

- Deployment status
- Runtime errors
- Service availability
- System behaviour

The monitoring system helps developers and operators identify failures, deployment issues, or runtime anomalies during application execution.

7. Feedback and Continuous Improvement Flow

The monitoring data generated by Jenkins and Kibana acts as a continuous feedback mechanism within the DevOps lifecycle.

Build failures, deployment issues, application errors, and runtime monitoring insights help developers identify areas requiring improvement.

This feedback loop supports:

- Faster debugging
- Continuous system improvement
- Early issue detection
- Improved deployment reliability
- Enhanced operational visibility

The overall DevOps pipeline therefore ensures automated integration, deployment, monitoring, and operational management of the Doctor Recommendation MLOps application suite.

Chapter 4

Version Control and CI/CD Automation with Jenkins

Effective version control and robust CI/CD automation are foundational to modern software development and MLOps. This chapter details the strategies employed for managing the project's source code and artifacts using Git and GitHub, and elaborates on the setup, configuration, and implementation of the Jenkins CI/CD pipeline designed to automate the lifecycle of all three Doctor Recommendation System microservices — the Frontend Gateway, the Speciality Predictor, and the Bandit Recommender.

1 Git Repository Structure and Management

The entire project, including application source code, ML model references, infrastructure-as-code configurations, and documentation, is managed within a single Git repository. This centralised approach facilitates collaboration, version tracking, and integration with the Jenkins CI/CD system.

1.1 Repository Organisation

The repository is structured with clearly separated directories for each microservice and DevOps configuration component, enhancing clarity and modularity:

- **main1/** — Contains the Bandit Recommender microservice. Key files include `app/server.py` (FastAPI application), `app/bandit_experience.py` (MultiArmedBanditExperience class implementing the epsilon-greedy policy), `app/bandit.joblib` (serialised bandit model), `requirements.txt`, and `Dockerfile`.
- **main2/** — Contains the Speciality Predictor microservice. Key files include `server.py` (Flask application), `specialist.py`, `specialist_description.csv`, `disease_description_updated_dataframe.csv`, `requirements.txt`, and `Dockerfile`.
- **app.py (root)** — The FastAPI Frontend Gateway application responsible for orchestrating communication between the backend microservices and rendering responses to users.
- **Ansible/** — Contains the deployment automation files including `deploy.yml` and `hosts.ini`.

- **Kubernetes/** — Contains Kubernetes deployment templates including:

- `bandit-deployment.yaml.j2`
- `speciality-deployment.yaml.j2`
- `frontend-deployment.yaml.j2`

The directory also contains ELK stack deployment manifests such as `elasticsearch.yaml` and `kibana.yaml`.

- **Jenkinsfile** — Declarative CI/CD pipeline definition used by Jenkins.
- **Dockerfile** — Root-level Dockerfile used for containerising the Frontend Gateway application.

This modular repository structure improves maintainability and allows each microservice to evolve independently while still being integrated through the DevOps pipeline.

1.2 Branching Strategy

The project primarily uses the `main` branch as the integration branch. Jenkins monitors this branch through SCM polling and automatically triggers the pipeline whenever changes are detected.

For larger collaborative environments, a feature-branch workflow could be adopted, where developers create separate branches for new features or bug fixes before merging into the `main` branch via pull requests. Such an approach would further improve code review practices and deployment stability.

2 Jenkins Setup and Configuration

Jenkins acts as the automation engine for the CI/CD workflow, orchestrating the build, push, image loading, and Kubernetes deployment of all Doctor Recommendation System microservices.

2.1 Jenkins Architecture

The project uses a Jenkins master-only architecture for simplicity. The Jenkins server is configured with access to:

- Docker CLI
- Minikube CLI
- `kubectl`
- Ansible

The Kubernetes configuration file is accessed using:

```
/var/lib/jenkins/.kube/config
```

This allows Jenkins to directly communicate with the Minikube Kubernetes cluster during deployment stages.

2.2 Key Jenkins Plugins Utilised

The following plugins are essential for the pipeline operation:

- **Pipeline Plugin** — Enables declarative pipeline definitions using Jenkinsfile.
- **Git Plugin** — Allows Jenkins to clone and poll the GitHub repository.
- **Credentials Plugin** — Securely stores Docker Hub credentials used during image push operations.

2.3 GitHub Integration: SCM Polling

The Jenkins pipeline uses SCM polling to monitor the GitHub repository for updates. The following trigger configuration is used:

```
triggers {  
    pollSCM('H/5 * * * *')  
}
```

This configuration instructs Jenkins to check the repository every five minutes for new commits. Whenever a change is detected on the `main` branch, the complete CI/CD pipeline is executed automatically.

3 Jenkins Pipeline Implementation (Jenkinsfile)

The complete CI/CD workflow is defined as code inside the `Jenkinsfile`. This approach ensures that the deployment process remains version-controlled, reproducible, and transparent.

3.1 Pipeline-as-Code with Jenkinsfile

The Jenkins pipeline defines global environment variables for Docker image names, Kubernetes configuration paths, and Ansible deployment files.

```
pipeline {  
    agent any  
  
    environment {  
        BANDIT_IMAGE      = "shrutimrao/bandit"  
        SPECIALITY_IMAGE  = "shrutimrao/speciality"  
        FRONTEND_IMAGE    = "shrutimrao/frontend"  
  
        ANSIBLE_INVENTORY = "Ansible/hosts.ini"  
        ANSIBLE_PLAYBOOK  = "Ansible/deploy.yml"  
    }  
  
    stages { ... }  
}
```

This configuration centralises all deployment variables and simplifies maintenance of the pipeline.

3.2 Stage 1: Checkout Code

The first stage confirms that Jenkins has successfully fetched the latest source code from GitHub.

```
stage('Checkout Code') {
    steps {
        echo "Code checked out."
    }
}
```

This stage ensures that the pipeline always operates on the latest committed version of the repository.

3.3 Stage 2: Build Docker Images

The second stage builds Docker images for all three microservices:

```
stage('Build Docker Images') {
    steps {
        script {
            sh "docker build -t ${BANDIT_IMAGE}:latest ./main1"
            sh "docker build -t ${SPECIALITY_IMAGE}:latest ./main2"
            sh "docker build -t ${FRONTEND_IMAGE}:latest ."
        }
    }
}
```

The images generated are:

- shrutimrao/bandit:latest
- shrutimrao/speciality:latest
- shrutimrao/frontend:latest

Separate Dockerfiles are used for each microservice to avoid dependency conflicts and minimise image sizes.

3.4 Stage 3: Push Docker Images to Docker Hub

After image creation, Jenkins authenticates with Docker Hub and pushes the images:

```
stage('Push Docker Images to DockerHub') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-credentials',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS')]) {
            sh '''
```

```

        docker push shrutimrao/bandit:latest
        docker push shrutimrao/speciality:latest
        docker push shrutimrao/frontend:latest
    """
}
}
}

```

Docker Hub acts as the central image repository for all container artifacts.

3.5 Stage 4: Load Images into Minikube

The images are then loaded directly into the Minikube image cache:

```

stage('Load Images into Minikube') {
    steps {
        sh """
            minikube image load ${BANDIT_IMAGE}:latest
            minikube image load ${SPECIALITY_IMAGE}:latest
            minikube image load ${FRONTEND_IMAGE}:latest
        """
    }
}

```

This ensures that Kubernetes pods can access locally built images without repeatedly downloading them from Docker Hub.

3.6 Stage 5: Deploy with Ansible

The final stage invokes the Ansible deployment playbook:

```

stage('Deploy with Ansible') {
    steps {
        sh """
            ansible-playbook -i ${ANSIBLE_INVENTORY}
                               ${ANSIBLE_PLAYBOOK}
        """
    }
}

```

The Ansible playbook performs the following tasks:

- Renders the Jinja2-templated Kubernetes manifests into concrete YAML files.
- Executes `kubectl apply` commands to deploy or update the Kubernetes resources.
- Verifies successful rollout completion using:

```
kubectl rollout status
```

- Deploys all three microservices:
 - Frontend Gateway
 - Speciality Predictor
 - Bandit Recommender

The Speciality Predictor deployment uses a longer rollout timeout because the BioDistillGPT-2 model requires additional startup time during initial deployment.

4 Automated Triggers and Notifications

The Jenkins pipeline operates automatically using SCM polling. Every detected commit triggers the entire deployment pipeline without manual intervention.

The Jenkinsfile also defines post-build actions for reporting deployment outcomes:

```
post {  
    success {  
        echo "Deployment successful!"  
    }  
  
    failure {  
        echo "Deployment failed."  
    }  
}
```

In production-grade systems, additional integrations such as email notifications, Slack alerts, or Microsoft Teams integration could be incorporated to provide real-time deployment feedback to development and operations teams.

This automated CI/CD workflow ensures that source code changes are consistently transformed into deployable Docker images and seamlessly deployed to the Kubernetes cluster with minimal manual effort.

Chapter 5

Containerization with Docker and Docker Compose

Containerization ensures that application services run in isolated, reproducible, and portable environments regardless of the underlying host machine. In the Doctor Recommendation system, Docker is used to package the three primary microservices — the Bandit Recommendation backend, the Speciality Predictor backend, and the Frontend Gateway — into separate container images. Each service uses its own Dockerfile tailored to its dependency requirements and runtime behaviour.

Unlike traditional monolithic deployments, this microservice-based containerization strategy enables each component to be independently built, deployed, scaled, and updated. The resulting Docker images are integrated into the Jenkins CI/CD pipeline and deployed to the Kubernetes cluster through Ansible automation.

1 Application Architecture and Containerization Strategy

The Doctor Recommendation System is divided into three independently containerized services:

Service	Directory	Framework	Port
Bandit Backend	main1/	FastAPI + Uvicorn	8000
Speciality Predictor	main2/	Flask	8080
Frontend Gateway	Root Directory	FastAPI + Uvicorn	5001

Table 5.1: Containerized Microservices in the Doctor Recommendation System

Each microservice contains its own:

- Source code
- Dependency definitions
- Runtime configuration

- Dockerfile

This separation improves modularity and prevents dependency conflicts between services. For example, the Speciality Predictor requires heavy NLP libraries such as PyTorch, Transformers, and spaCy, whereas the Frontend Gateway requires only lightweight FastAPI dependencies.

2 Dockerfile for the Bandit Recommendation Service

The Bandit Recommendation service is located inside the `main1/` directory and is implemented using FastAPI.

2.1 Dockerfile Structure

```
FROM python:3.12-slim

WORKDIR /code

COPY ./requirements.txt /code/requirements.txt
COPY ./app/bandit_experience.py /code/

RUN pip install --no-cache-dir -r /code/requirements.txt

COPY ./app /code/app
COPY . .

EXPOSE 8000

CMD ["python", "-m", "uvicorn",
     "app.server:app",
     "--host", "0.0.0.0",
     "--port", "8000"]
```

2.2 Explanation of Key Instructions

- **FROM python:3.12-slim**
Uses a lightweight Python 3.12 base image to reduce image size while maintaining compatibility with the FastAPI application.
- **WORKDIR /code**
Sets the working directory inside the container.
- **COPY requirements.txt**
Copies dependency definitions before copying the full application source code. This improves Docker layer caching efficiency.
- **RUN pip install**
Installs the Python dependencies including:
 - fastapi

- uvicorn
- numpy
- pandas
- joblib
- pydantic
- **COPY ./app /code/app**
Copies the application source code including:
 - server.py
 - bandit_experience.py
 - bandit.joblib
- **EXPOSE 8000**
Documents that the container listens on port 8000.
- **CMD**
Starts the FastAPI application using the Uvicorn ASGI server.

The pre-trained `bandit.joblib` model is bundled directly inside the image. This makes the container self-contained and simplifies deployment.

3 Dockerfile for the Speciality Predictor Service

The Speciality Predictor service is located inside the `main2/` directory and uses Flask along with transformer-based NLP inference.

3.1 Dockerfile Structure

```
FROM python:3.10-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080
```

```
ENV FLASK_APP=server.py
ENV FLASK_RUN_HOST=0.0.0.0

CMD ["python", "server.py"]
```

3.2 Explanation of Key Instructions

- **FROM python:3.10-slim**

Python 3.10 is used due to compatibility requirements of NLP libraries such as spaCy and PyTorch.

- **RUN apt-get install**

Installs build tools including:

- gcc
- g++
- build-essential

These tools are required because some Python packages contain native C extensions.

- **RUN pip install**

Installs heavy NLP dependencies including:

- transformers
- torch
- spacy
- scikit-learn
- pandas

- **ENV FLASK_APP**

Configures Flask runtime environment variables.

- **CMD ["python", "server.py"]**

Launches the Flask server.

At startup, this service downloads and loads the BioDistillGPT-2 transformer model from HuggingFace Hub. This results in slower startup times compared to the other microservices.

4 Dockerfile for the Frontend Gateway

The Frontend Gateway acts as the orchestrator between the user interface and the backend microservices.

4.1 Dockerfile Structure

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5001

CMD ["uvicorn", "app:app",
     "--host", "0.0.0.0",
     "--port", "5001"]
```

4.2 Explanation of Key Instructions

- **FROM python:3.10-slim**
Uses a lightweight Python image suitable for FastAPI applications.
- **COPY requirements.txt before COPY . .**
Improves Docker cache utilisation.
- **EXPOSE 5001**
Defines port 5001 as the application port.
- **CMD**
Launches the FastAPI gateway using Uvicorn.

The Frontend Gateway communicates with the backend services using Kubernetes internal DNS names provided through environment variables.

5 Building Docker Images

Docker images are built automatically during the Jenkins pipeline execution using the following commands:

```
docker build -t shrutimrao/bandit:latest ./main1

docker build -t shrutimrao/speciality:latest ./main2

docker build -t shrutimrao/frontend:latest .
```

Each image is tagged using the `latest` tag and stored under the `shrutimrao` Docker Hub namespace.

The Docker build process creates isolated execution environments containing:

- Application source code
- Runtime dependencies
- ML model files
- Startup commands

6 Pushing Docker Images to Docker Hub

After successful image creation, Jenkins authenticates with Docker Hub using securely stored credentials and pushes the images.

```
docker push shrutimrao/bandit:latest
```

```
docker push shrutimrao/speciality:latest
```

```
docker push shrutimrao/frontend:latest
```

Docker Hub serves as the central image repository for the CI/CD pipeline. Jenkins uses the Credentials Plugin to securely inject Docker Hub usernames and passwords during pipeline execution.

7 Loading Images into Minikube

Since the project uses a local Minikube Kubernetes cluster, the Docker images must also be loaded into Minikube's internal image cache.

```
minikube image load shrutimrao/bandit:latest
```

```
minikube image load shrutimrao/speciality:latest
```

```
minikube image load shrutimrao/frontend:latest
```

This step ensures that Kubernetes pods can use the locally built images without repeatedly downloading them from Docker Hub.

The Kubernetes deployments use:

```
imagePullPolicy: IfNotPresent
```

This configuration allows Kubernetes to reuse locally available images whenever possible.

8 Multi-Container Service Communication

The three microservices communicate internally through Kubernetes networking and DNS-based service discovery.

The Frontend Gateway accesses the backend services using:

```
http://speciality:8080/predict
```

```
http://bandit:8000/recommend/
```

These URLs are injected into the Frontend container as Kubernetes environment variables. Only the Frontend service is externally exposed through a NodePort service. The backend services remain internal using ClusterIP services, improving security by preventing direct external access.

Chapter 6

Configuration Management with Ansible

Configuration management is the practice of ensuring that software environments are defined, version-controlled, and reproducible rather than configured manually each time. In this project, Ansible plays a focused role in automating Kubernetes deployments for the Doctor Recommendation microservices application. Unlike traditional infrastructure provisioning scenarios, Ansible here acts as a deployment orchestration layer between Jenkins and Kubernetes.

The Ansible configuration automates:

- Rendering Kubernetes manifests from Jinja2 templates
- Applying Kubernetes Deployment and Service resources
- Verifying rollout completion
- Managing deployment consistency across services

All Ansible configurations are maintained inside the `Ansible/` directory of the project repository.

1 Role of Ansible in This Project

The role of Ansible in this project is centred around deployment automation and Kubernetes manifest management.

The project contains three Kubernetes manifest templates:

- `bandit-deployment.yaml.j2`
- `speciality-deployment.yaml.j2`
- `frontend-deployment.yaml.j2`

These manifests are not static YAML files. Instead, they are parameterised Jinja2 templates containing placeholders for Docker image names and tags.

Ansible performs the following operations:

1. Renders the Jinja2 templates into concrete Kubernetes YAML manifests
2. Substitutes Docker image names and tags dynamically
3. Applies the manifests to the Minikube Kubernetes cluster

4. Verifies deployment rollout completion

This approach separates deployment logic from the Jenkins pipeline and centralises Kubernetes deployment operations within Ansible.

2 Ansible Inventory Management

Ansible uses an inventory file to define the hosts on which playbooks operate. The inventory file used in this project is:

```
[master]
localhost ansible_connection=local
```

The inventory defines a single host group named `master`. The host is `localhost`, and the directive:

```
ansible_connection=local
```

instructs Ansible to execute all tasks locally instead of using SSH. This configuration is appropriate because:

- Kubernetes runs locally through Minikube
- `kubectl` commands are executed on the Jenkins host
- No remote servers are provisioned

The playbook later references this group using:

```
hosts: master
```

3 Ansible Playbook: `deploy.yml`

The core deployment automation logic is implemented inside:

```
Ansible/deploy.yml
```

The playbook structure is shown below:

```
---
- hosts: master
  become: no

  vars:
    bandit_image: "shrutimrao/bandit"
    speciality_image: "shrutimrao/speciality"
    frontend_image: "shrutimrao/frontend"

  tasks:
    ...
```

The playbook uses:

- `hosts:` `master` to target the local machine
- `become:` `no` because elevated privileges are unnecessary
- Play-level variables for Docker image repository names

The image tags are supplied dynamically from Jenkins using:

```
-e bandit_tag=latest
-e speciality_tag=latest
-e frontend_tag=latest
```

This allows the same playbook to deploy different image versions without modifying the playbook source code.

4 Rendering Kubernetes Manifests from Jinja2 Templates

The first phase of the playbook renders Kubernetes manifests using Ansible's `template` module.

Example task:

```
- name: "Update Kubernetes manifests with latest Bandit tag"
  template:
    src: "../Kubernetes/bandit-deployment.yaml.j2"
    dest: "/tmp/bandit-deployment.yaml"
```

The `template` module performs variable substitution on Jinja2 expressions such as:

```
image: {{ bandit_image }}:{{ bandit_tag }}
```

After rendering, the output becomes:

```
image: shrutimrao/bandit:latest
```

The rendered manifests are temporarily stored inside the `/tmp/` directory before deployment. This process is repeated for:

- Bandit Recommendation service
- Speciality Predictor service
- Frontend Gateway service

5 Applying Kubernetes Deployments

After rendering the manifests, Ansible applies them to Kubernetes using `kubectl apply`.
Example:

```
- name: "Apply Bandit Deployment"
  command: kubectl apply -f /tmp/bandit-deployment.yaml
```

The same process is repeated for:

- `speciality-deployment.yaml`
- `frontend-deployment.yaml`

The command:

```
kubectl apply
```

is declarative in nature. Kubernetes compares the desired cluster state defined in the YAML file with the current cluster state and applies only the necessary changes.

This enables:

- Rolling updates
- Controlled deployment changes
- Declarative infrastructure management

6 Deployment Rollout Verification

After deployment, Ansible verifies rollout completion using:

```
kubectl rollout status
```

Example:

```
- name: "Verify Rollout Status (Bandit)"
  command: kubectl rollout status deployment/bandit-deployment --timeout=
```

The rollout verification phase ensures:

- Pods are successfully created
- Containers become healthy
- Readiness probes succeed
- Deployments complete before the pipeline proceeds

Different timeout values are configured for different services.

- Bandit Deployment: 180 seconds
- Frontend Deployment: 180 seconds
- Speciality Deployment: 300 seconds

The Speciality Predictor requires a longer timeout because the BioDistillGPT-2 transformer model is downloaded and loaded during startup.

7 Frontend Service Environment Variables

The Frontend deployment template injects Kubernetes environment variables into the container.
Example:

```
env:
  - name: SPECIALITY_URL
    value: "http://speciality:8080/predict"

  - name: BANDIT_URL
    value: "http://bandit:8000/recommend/"
```

These URLs use Kubernetes internal DNS-based service discovery.
This allows the Frontend Gateway to communicate with:

- the Speciality Predictor service
- the Bandit Recommendation service

without requiring hardcoded IP addresses.

8 Use of Variables and Jinja2 Templates

Ansible uses Jinja2 templating to separate deployment configuration from Kubernetes manifest structure.

Playbook variables define Docker repositories:

```
vars:
  bandit_image: "shrutimrao/bandit"
  speciality_image: "shrutimrao/speciality"
  frontend_image: "shrutimrao/frontend"
```

Jenkins supplies image tags dynamically using:

```
-e bandit_tag=latest
```

The Kubernetes templates then combine both values:

```
image: {{ bandit_image }}:{{ bandit_tag }}
```

This design improves:

- Reusability
- Deployment flexibility
- Environment independence

9 Idempotency Considerations

Idempotency is a core property of Ansible automation.

The `template` module is idempotent because files are only updated when their rendered contents change.

Similarly:

```
kubectl apply
```

is declarative and idempotent at the Kubernetes level. Reapplying the same manifest multiple times does not create duplicate resources.

This allows deployments to be repeated safely without introducing inconsistent cluster states.

10 Relationship Between Jenkins and Ansible

Jenkins invokes Ansible during the deployment phase of the CI/CD pipeline.

The Jenkins stage executes:

```
ansible-playbook -i Ansible/hosts.ini Ansible/deploy.yml \
    -e bandit_tag=latest \
    -e speciality_tag=latest \
    -e frontend_tag=latest
```

Jenkins itself does not directly interact with Kubernetes.

Instead:

1. Jenkins triggers the Ansible playbook
2. Ansible renders Kubernetes manifests
3. Ansible applies deployments using `kubectl`
4. Ansible verifies rollout completion
5. Jenkins receives the final deployment status

This separation improves modularity and keeps deployment logic independent from CI orchestration logic.

11 Summary

Ansible serves as the deployment orchestration layer of the Doctor Recommendation System DevOps pipeline. Through Jinja2 templating, Kubernetes manifest rendering, deployment automation, and rollout verification, Ansible ensures that all three microservices are deployed consistently and reproducibly to the Minikube Kubernetes cluster.

The integration between Jenkins, Ansible, Docker, and Kubernetes creates a streamlined CI/CD workflow capable of automatically deploying updated application versions while maintaining deployment reliability and operational consistency.

Chapter 7

Orchestration and Deployment with Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform used to automate the deployment, management, and scaling of containerised applications. In the Doctor Recommendation system, Kubernetes is used to orchestrate the deployment of the three microservices that make up the application: the Frontend Gateway, the Speciality Predictor backend, and the Bandit Recommendation backend.

The Kubernetes configuration files are located inside the `Kubernetes/` directory of the project repository. The deployment process is automated through Jenkins and Ansible, which render the Kubernetes Jinja2 templates and apply them to the Minikube cluster.

1 Kubernetes Cluster Overview

The project uses Minikube as the Kubernetes environment for local deployment and testing. Minikube provides a lightweight single-node Kubernetes cluster suitable for development and experimentation.

The Kubernetes cluster hosts:

- Frontend Gateway service
- Speciality Predictor service
- Bandit Recommendation service

The Jenkins CI/CD pipeline deploys the application to the cluster through Ansible automation. The deployment workflow is:

1. Jenkins builds Docker images
2. Images are pushed to Docker Hub
3. Images are loaded into Minikube
4. Ansible renders Kubernetes manifests
5. Kubernetes deployments and services are applied
6. Rollout status is verified

2 Application Architecture in Kubernetes

The Doctor Recommendation System application follows a microservice-based architecture composed of three independently deployed services.

- Frontend Gateway
- Speciality Predictor
- Bandit Recommendation Service

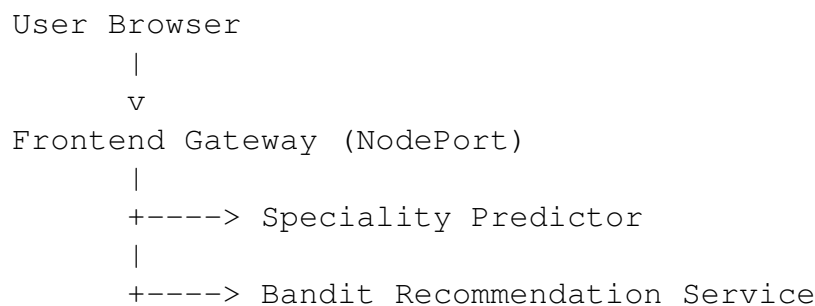
The Frontend Gateway acts as the entry point for user requests. Users interact with the frontend through a web interface exposed via a Kubernetes NodePort service.

The Frontend Gateway internally communicates with:

- the Speciality Predictor service
- the Bandit Recommendation service

using Kubernetes internal DNS-based service discovery.

The communication flow is:



This architecture separates NLP prediction logic, recommendation logic, and frontend interaction into independently manageable services.

3 Bandit Recommendation Kubernetes Deployment

The file:

```
Kubernetes/bandit-deployment.yaml.j2
```

defines the Deployment and Service resources for the Bandit Recommendation backend.

3.1 Bandit Deployment Configuration

The deployment uses:

- `replicas: 1`
- FastAPI + Uvicorn container
- Port 8000

- Readiness and liveness probes

The Docker image is dynamically injected using Jinja2 variables:

```
image: {{ bandit_image }}:{{ bandit_tag }}
```

Ansible renders these variables during deployment.

The deployment uses:

```
imagePullPolicy: IfNotPresent
```

because Docker images are already loaded into the Minikube image cache during the Jenkins pipeline.

3.2 Bandit Readiness and Liveness Probes

The Bandit service uses HTTP-based health checks:

```
readinessProbe:
  httpGet:
    path: /docs
    port: 8000
```

The FastAPI `/docs` endpoint becomes available only after the application and the serialized `bandit.joblib` model are fully loaded.

The liveness probe also uses the same endpoint to ensure the container remains responsive during runtime.

These probes improve deployment reliability and allow Kubernetes to restart failed containers automatically.

3.3 Bandit Service Configuration

The Kubernetes Service configuration exposes the backend internally using:

```
type: ClusterIP
```

The service is reachable within the cluster using:

```
http://bandit:8000
```

Only internal cluster communication is allowed because external users interact exclusively through the Frontend Gateway.

4 Speciality Predictor Kubernetes Deployment

The file:

```
Kubernetes/speciality-deployment.yaml.j2
```

defines the Deployment and Service resources for the Speciality Predictor backend.

4.1 Speciality Deployment Configuration

The deployment uses:

- Flask backend
- BioDistillGPT-2 model
- Port 8080
- TCP-based health checks

The deployment image is parameterised using:

```
image: {{ speciality_image }}:{{ speciality_tag }}
```

The deployment also uses:

```
imagePullPolicy: IfNotPresent
```

to utilise locally cached Minikube images.

4.2 TCP Health Checks

Unlike the FastAPI services, the Speciality Predictor uses TCP socket probes.

```
readinessProbe:  
  tcpSocket:  
    port: 8080
```

The transformer model requires substantial startup time because the HuggingFace model is downloaded and loaded during container initialisation.

Using TCP probes avoids premature health check failures while the Flask application is still loading the model.

The deployment therefore uses:

- longer initial delays
- longer rollout timeout values

to accommodate model initialisation.

4.3 Speciality Service Configuration

The service is exposed internally through:

```
type: ClusterIP
```

The Frontend Gateway communicates with the service using:

```
http://speciality:8080/predict
```

through Kubernetes internal DNS.

5 Frontend Gateway Kubernetes Deployment

The file:

```
Kubernetes/frontend-deployment.yaml.j2
```

defines the Deployment and Service resources for the Frontend Gateway.

5.1 Frontend Deployment Configuration

The Frontend Gateway uses:

- FastAPI
- Uvicorn
- Port 5001

The deployment injects environment variables directly into the container:

```
env:  
  - name: SPECIALITY_URL  
    value: "http://speciality:8080/predict"  
  
  - name: BANDIT_URL  
    value: "http://bandit:8000/recommend/"
```

These environment variables allow the frontend service to communicate with the backend microservices without hardcoding IP addresses.

5.2 Frontend Health Checks

The Frontend Gateway uses HTTP-based probes against the root endpoint:

/

The readiness probe ensures:

- FastAPI is running
- Templates are loaded
- The service can accept requests

The liveness probe ensures failed containers are automatically restarted.

5.3 Frontend Service Exposure

Unlike the backend services, the Frontend Gateway uses:

```
type: NodePort
```

This exposes the application externally through the Minikube node IP and NodePort.

Users access the application using:

```
minikube service frontend --url
```

The Frontend Gateway therefore acts as the single public entry point for the Doctor Recommendation system.

6 Service Discovery and Inter-Service Communication

Kubernetes automatically provides internal DNS-based service discovery. The services communicate using:

- `http://speciality:8080`
- `http://bandit:8000`

instead of fixed IP addresses.

The Frontend Gateway performs the following sequence:

1. Sends symptom text to the Speciality Predictor
2. Receives predicted medical specialties
3. Sends the specialties to the Bandit Recommendation service
4. Receives ranked doctor recommendations
5. Returns results to the user

This communication architecture keeps services loosely coupled and independently deployable.

7 Rolling Update Deployment Strategy

The Kubernetes deployments use the default rolling update strategy. During deployment updates:

1. New pods are created
2. Readiness probes are validated
3. Old pods are terminated

This ensures minimal downtime during application updates. The rollout process is verified through:

```
kubectl rollout status
```

commands executed by Ansible.

8 Health Check Design Summary

Each microservice uses health checks tailored to its runtime behaviour.

Service	Probe Type	Endpoint	Purpose
Bandit Backend	HTTP GET	/docs	Verify FastAPI startup and model loading
Speciality Predictor	TCP Socket	Port 8080	Handle slow transformer model startup
Frontend Gateway	HTTP GET	/	Verify frontend availability

Table 7.1: Health Check Strategy for Kubernetes Services

The use of service-specific health checks improves deployment stability and allows Kubernetes to automatically recover unhealthy containers.

9 Summary

Kubernetes provides the orchestration foundation for the Doctor Recommendation system. Through Deployments, Services, health checks, rolling updates, and internal DNS-based communication, Kubernetes enables reliable deployment and management of the application's microservice architecture.

Combined with Jenkins, Docker, and Ansible, Kubernetes forms the deployment and orchestration layer of the complete DevOps pipeline.

Chapter 8

Monitoring and Logging with Elasticsearch and Kibana

Monitoring and logging are essential components of modern DevOps and MLOps systems. They provide operational visibility into deployed applications, enable troubleshooting, assist in identifying runtime failures, and help monitor overall system behaviour.

In the Doctor Recommendation system, centralized logging and monitoring are implemented using Elasticsearch and Kibana deployed inside the Kubernetes cluster. Unlike a complete ELK stack deployment, this project uses only Elasticsearch and Kibana. Logstash is not used in the current implementation.

The logging infrastructure is deployed within a dedicated Kubernetes namespace named:

`logging`

This separation isolates monitoring infrastructure from the main application workloads.

1 Role of Monitoring in the Doctor Recommendation System

The Doctor Recommendation System platform consists of multiple interacting microservices:

- Frontend Gateway
- Speciality Predictor
- Bandit Recommendation Service

Each service generates runtime logs during execution. Monitoring these logs is important for:

- Detecting application failures
- Identifying deployment issues
- Observing runtime behaviour
- Debugging inter-service communication
- Monitoring container startup and health

The monitoring infrastructure provides centralized visibility into the Kubernetes environment and deployed services.

2 Logging Namespace Architecture

The Elasticsearch and Kibana deployments are isolated inside the Kubernetes namespace:

```
logging
```

The namespace is created using:

```
kubectl create namespace logging
```

The logging namespace contains:

- Elasticsearch Deployment and Service
- Kibana Deployment and Service

Separating logging infrastructure into its own namespace improves:

- resource organization
- operational isolation
- deployment clarity
- infrastructure management

3 Elasticsearch Deployment

The Elasticsearch configuration is defined in:

```
Kubernetes/logging/elasticsearch.yaml
```

Elasticsearch acts as the centralized log storage and indexing engine of the monitoring infrastructure.

3.1 Elasticsearch Deployment Configuration

The deployment uses the official Elasticsearch Docker image:

```
docker.elastic.co/elasticsearch/elasticsearch:7.17.10
```

The deployment configuration includes:

- Single-node Elasticsearch setup
- ClusterIP service exposure
- JVM memory tuning
- Disabled X-Pack security

The deployment defines:

```
discovery.type: single-node
```

because the project uses a single-node Minikube Kubernetes cluster.

Without this configuration, Elasticsearch would continuously wait for additional cluster nodes and fail to initialize correctly.

3.2 Memory Configuration

Elasticsearch is memory intensive. The deployment therefore restricts JVM heap allocation using:

```
ES_JAVA_OPTS: "-Xms512m -Xmx512m"
```

This configuration:

- limits Elasticsearch memory consumption
- prevents Minikube resource exhaustion
- improves local cluster stability

This is particularly important because Minikube runs on limited local system resources.

3.3 Security Configuration

The deployment disables X-Pack security using:

```
xpack.security.enabled: "false"
```

This simplifies the local development environment by avoiding:

- TLS certificate setup
- authentication configuration
- secure transport configuration

This configuration is suitable for development and demonstration environments but would not be recommended for production systems.

3.4 Elasticsearch Service Configuration

The Elasticsearch service uses:

```
type: ClusterIP
```

This ensures Elasticsearch remains internally accessible only within the Kubernetes cluster. The service exposes:

```
port: 9200
```

and is accessed internally by Kibana using Kubernetes DNS.

4 Kibana Deployment

The Kibana configuration is defined in:

```
Kubernetes/logging/kibana.yaml
```

Kibana provides the visualization and dashboard interface for monitoring Elasticsearch data.

4.1 Kibana Deployment Configuration

The deployment uses the official Kibana Docker image:

```
docker.elastic.co/kibana/kibana:7.17.10
```

The Kibana version matches the Elasticsearch version to maintain compatibility.
The deployment exposes:

```
containerPort: 5601
```

which is Kibana's default web interface port.

4.2 Connecting Kibana to Elasticsearch

Kibana connects to Elasticsearch using the environment variable:

```
ELASTICSEARCH_HOSTS=http://elasticsearch.logging.svc.cluster.local:9200
```

This uses Kubernetes fully qualified DNS naming:

```
<service>.<namespace>.svc.cluster.local
```

The fully qualified DNS name is necessary because the services operate inside the dedicated logging namespace.

This configuration allows Kibana to discover Elasticsearch automatically through Kubernetes networking.

4.3 Kibana Service Exposure

Unlike Elasticsearch, Kibana is externally accessible using:

```
type: NodePort
```

This allows developers to access the Kibana dashboard from a browser outside the Kubernetes cluster.

The Kibana access URL can be obtained using:

```
minikube service kibana -n logging --url
```

This command returns the Minikube node IP and dynamically assigned NodePort.

5 Centralized Logging Workflow

The monitoring workflow of the Doctor Recommendation system is based on centralized log visibility.

The process operates as follows:

1. Application microservices generate runtime logs
2. Kubernetes captures container stdout/stderr streams
3. Elasticsearch stores and indexes log information
4. Kibana visualizes the indexed data

This centralized approach simplifies:

- debugging
- operational monitoring
- failure analysis
- deployment troubleshooting

6 Application Monitoring Use Cases

The monitoring infrastructure supports several operational tasks within the Doctor Recommendation platform.

6.1 Deployment Monitoring

Kibana can be used to observe:

- pod startup failures
- deployment rollout issues
- container restart loops
- application exceptions

This is especially useful for the Speciality Predictor service, which requires additional startup time because of transformer model loading.

6.2 Inter-Service Communication Monitoring

The Frontend Gateway communicates with:

- the Speciality Predictor service
- the Bandit Recommendation service

Failures in these HTTP interactions can be identified using runtime logs visible through Kibana dashboards.

This improves debugging of:

- timeout failures
- connection issues
- invalid API responses
- internal service failures

6.3 Container Health Monitoring

The Kubernetes liveness and readiness probes generate events when containers become unhealthy.

These events help identify:

- failed health checks
- startup delays
- application crashes
- unstable deployments

The monitoring system therefore assists in maintaining deployment reliability.

7 Namespace Separation and Infrastructure Isolation

The project separates application services and monitoring infrastructure into different Kubernetes namespaces.

Component	Namespace
Frontend Gateway	default
Bandit Recommendation Service	default
Speciality Predictor Service	default
Elasticsearch	logging
Kibana	logging

Table 8.1: Namespace Separation in the Doctor Recommendation System Kubernetes Cluster

This separation improves:

- infrastructure organization
- operational management
- deployment clarity
- service isolation

8 Limitations and Future Improvements

The current monitoring implementation focuses on basic centralized logging and visualization. Several improvements could further enhance observability:

- **Logstash Integration**

The current implementation does not use Logstash. Future implementations could integrate Logstash for advanced log processing and pipeline transformations.

- **Persistent Storage**

Elasticsearch currently uses ephemeral storage. Persistent Volumes could improve data durability.

- **Metrics Monitoring**

The system currently focuses primarily on logs. Prometheus and Grafana could be integrated for infrastructure and application metrics monitoring.

- **Alerting Systems**

Alerting mechanisms such as email notifications or Slack integration could improve operational response to failures.

9 Summary

Elasticsearch and Kibana provide centralized monitoring and logging capabilities for the Doctor Recommendation System platform. Through namespace isolation, Kubernetes service discovery, centralized log visualization, and operational monitoring, the monitoring infrastructure improves deployment visibility and simplifies debugging of the microservice-based healthcare recommendation system.

Integrated with Docker, Jenkins, Ansible, and Kubernetes, Elasticsearch and Kibana complete the operational observability layer of the overall DevOps and MLOps pipeline.

Chapter 9

System Demonstration, Testing, and Evaluation

1 Overview of System Execution

After successful implementation of the MLOps pipeline, the Doctor Recommendation System was deployed and executed on the Kubernetes cluster using Jenkins-driven CI/CD automation. The final system demonstrates successful integration of machine learning inference, microservice communication, automated deployment, container orchestration, and centralized monitoring.

The deployed application consists of three primary services:

- Frontend Gateway Service
- Medical Specialty Prediction Service
- Doctor Recommendation Service

The pipeline automatically performs source code checkout, Docker image generation, image publishing to Docker Hub, loading images into Minikube, and deployment using Ansible automation.

2 Successful Jenkins Pipeline Execution

The Jenkins CI/CD pipeline was executed successfully with all stages completing without errors. The pipeline integrates automated deployment operations and Kubernetes orchestration into a unified workflow.

The implemented Jenkins stages include:

1. Declarative SCM Checkout
2. Source Code Checkout
3. Docker Image Build
4. Push Images to Docker Hub

5. Load Images into Minikube
6. Kubernetes Deployment using Ansible
7. Post-build Cleanup Actions

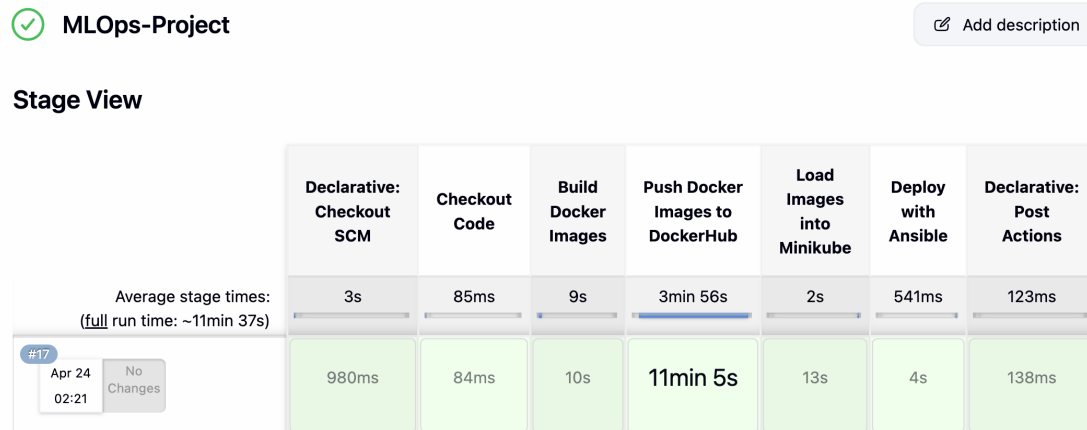


Figure 9.1: Successful Jenkins Pipeline Execution

3 Verifying Docker Image Availability on Docker Hub

After successful completion of the Docker image build and publishing stages, all three containerized microservices of the Doctor Recommendation System application were uploaded to Docker Hub.

The uploaded images correspond to:

- Frontend Gateway Service
- Speciality Prediction Service
- Bandit Recommendation Service

Publishing the images to Docker Hub enables centralized image storage and simplifies deployment across Kubernetes environments. The container registry acts as the distribution platform from which the deployment pipeline retrieves application images during orchestration and roll-out operations.

The successful availability of all generated Docker images on Docker Hub confirms that the Jenkins CI/CD pipeline correctly performed image tagging, authentication, and registry upload operations for all microservices.

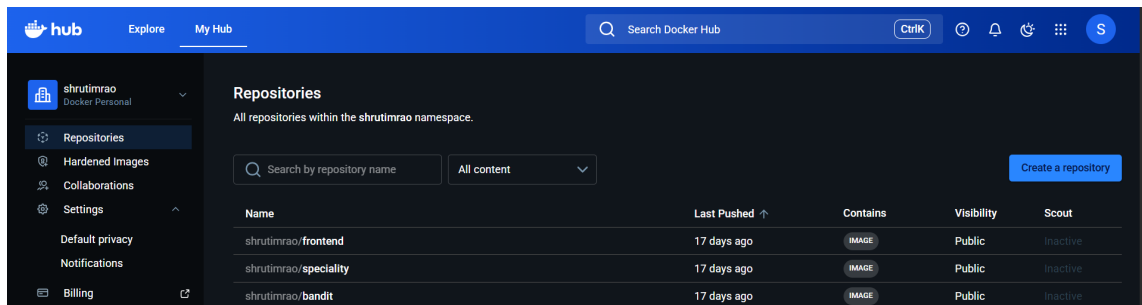


Figure 9.2: Docker Hub Repository Containing Uploaded Doctor Recommendation System Microservice Images

4 Frontend User Interface

The deployed frontend application provides a simple web-based interface where users can enter symptom descriptions for prediction and recommendation generation.

The frontend accepts natural language symptom descriptions such as:

- Headache
- Backpain
- Vomiting
- Fever

The request is then forwarded internally to the Speciality Predictor microservice for inference processing.

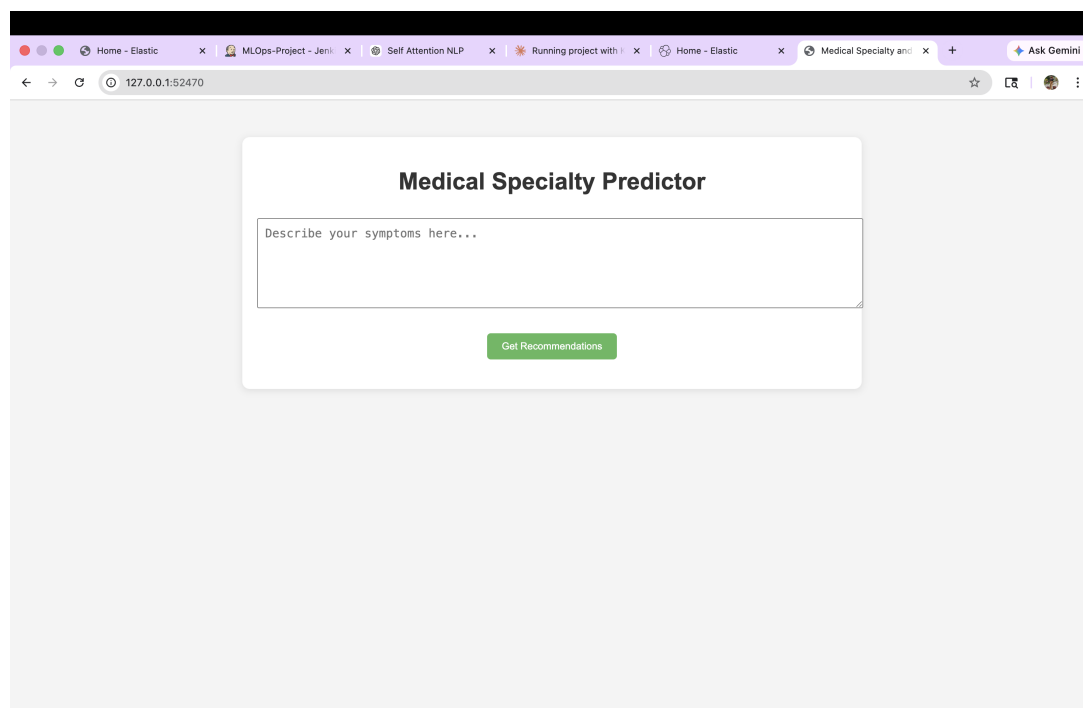


Figure 9.3: Frontend Interface for Symptom Input

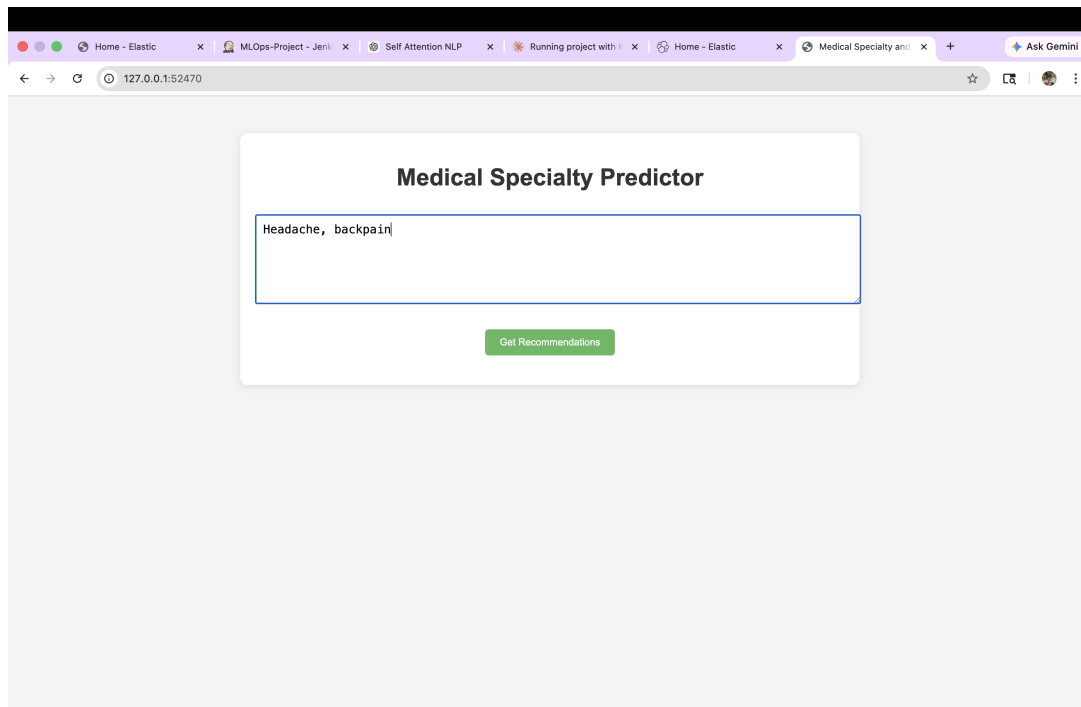


Figure 9.4: Symptom Input by user

5 Integrated Specialty Prediction and Doctor Recommendation

After symptom submission, the Frontend Gateway communicates with the Speciality Predictor microservice, which uses the BioDistillGPT-2 NLP model to predict relevant medical specialties.

The predicted specialties are then forwarded internally to the Bandit Recommendation microservice, which applies the epsilon-greedy Multi-Armed Bandit algorithm to generate ranked doctor recommendations.

The final output page therefore displays:

- Input symptom description
- Predicted medical specialties
- Ranked doctor recommendations
- Doctor ratings
- Doctor experience information

The complete workflow executes automatically within a single request-response cycle, demonstrating successful communication between independently deployed Kubernetes microservices.

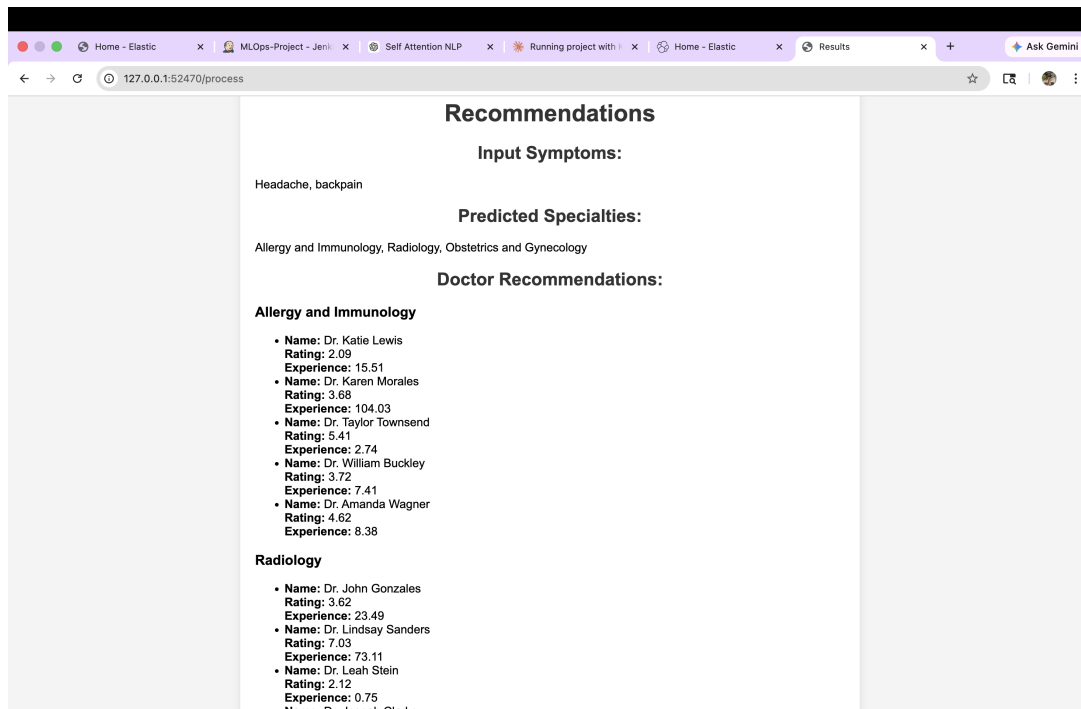


Figure 9.5: Integrated Specialty Prediction and Doctor Recommendation Results

6 Elastic Stack Monitoring Dashboard

The Elastic Stack was successfully deployed and integrated into the Kubernetes cluster to provide centralized monitoring, observability, and log management capabilities for the Doctor Recommendation System MLOps platform. Kibana was used as the primary visualization and analytics interface for monitoring application activity, deployment behaviour, and runtime logs generated by the Kubernetes-based microservices.

The Kibana home interface provides access to multiple monitoring and management components within the Elastic Stack environment, including analytics dashboards, observability tools, system monitoring utilities, and operational inspection services. This integration enables developers and administrators to monitor application behaviour and inspect deployment activities across the deployed microservices.

The monitoring platform supports:

- Centralized log visualization
- Application and infrastructure monitoring
- Analytics and dashboard management
- Observability and operational inspection

The Kibana home dashboard shown below confirms the successful deployment and accessibility of the Elastic monitoring environment within the Kubernetes cluster.

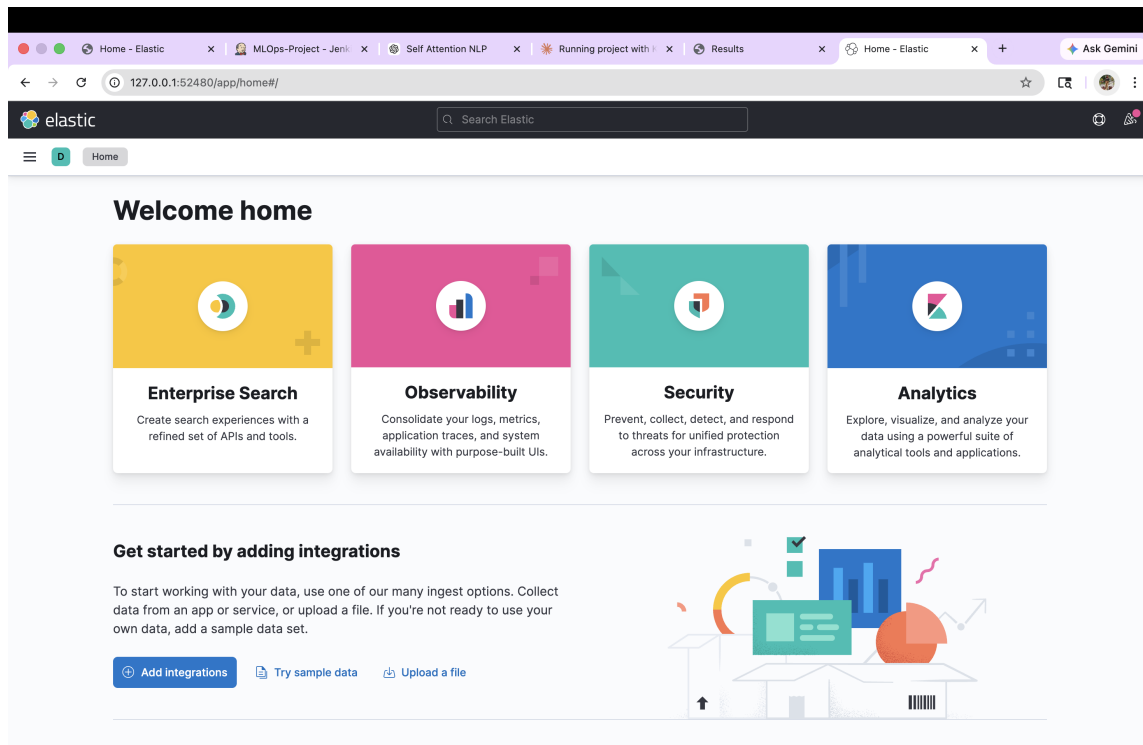


Figure 9.6: Elastic Home Dashboard

7 Elastic Observability Integration

The Elastic observability platform was integrated with the deployed microservices to support centralized inspection of runtime activities and deployment behaviour across the Doctor Recommendation System.

Logs generated by the Frontend Gateway, Speciality Predictor service, and Bandit Recommendation service can be collected, analysed, and visualized through the Elastic analytics interface. This enables continuous visibility into system execution, deployment status, service activity, and operational health within the Kubernetes environment.

The observability and analytics platform provides features for:

- Runtime monitoring of deployed services
- Inspection of Kubernetes-generated logs
- Infrastructure and deployment analytics
- Monitoring of application activity and health

The Elastic Analytics dashboard shown below verifies the successful integration of centralized monitoring and observability capabilities into the overall MLOps workflow.

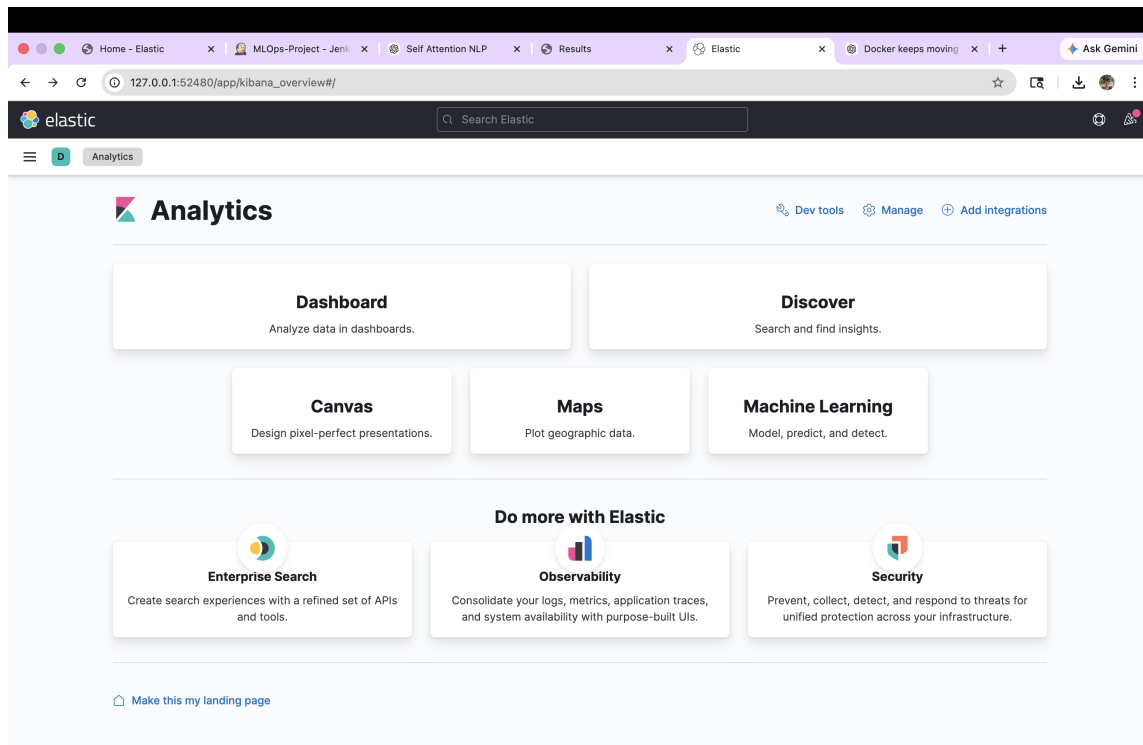


Figure 9.7: Elastic Analytics Dashboard

8 System Validation

The final deployed system validates the successful implementation of:

- CI/CD automation using Jenkins
- Containerization using Docker
- Kubernetes orchestration
- Infrastructure automation using Ansible
- NLP inference using BioDistillGPT-2
- Multi-Armed Bandit recommendation logic
- Centralized monitoring using Elastic Stack

The deployment demonstrates a complete end-to-end MLOps pipeline for scalable medical recommendation applications.

Chapter 10

Conclusion and Future Scope

1 Conclusion

This project successfully implemented an end-to-end MLOps-enabled Doctor Recommendation System using a microservice-based architecture deployed on Kubernetes.

The system integrates machine learning inference, recommendation algorithms, containerization, CI/CD automation, orchestration, and centralized monitoring into a unified deployment workflow.

The developed platform consists of three independently deployable services:

- Frontend Gateway
- Medical Specialty Prediction Service
- Doctor Recommendation Service

The Speciality Predictor service uses the BioDistillGPT-2 transformer model for NLP-based medical specialty prediction, while the recommendation engine applies an epsilon-greedy Multi-Armed Bandit algorithm to dynamically rank doctors.

Docker containerization enabled reproducible deployments and service isolation, while Kubernetes provided orchestration, scaling, and deployment management capabilities.

The Jenkins CI/CD pipeline automated the complete deployment workflow including:

- Source code retrieval
- Docker image generation
- Image publishing
- Minikube image loading
- Kubernetes deployment using Ansible

Elastic Stack integration further enabled centralized observability and runtime monitoring of deployed services.

Overall, the project demonstrates how modern MLOps practices can be applied to deploy scalable and maintainable AI-powered healthcare applications.

2 Future Scope

Several enhancements can further improve the scalability, intelligence, and usability of the Doctor Recommendation System platform.

Future improvements include:

- Integration of real hospital and doctor datasets
- Authentication and role-based access control
- GPU-enabled inference acceleration
- Real-time patient appointment scheduling
- Deployment on cloud Kubernetes platforms such as AWS EKS or Google Kubernetes Engine
- Advanced recommendation algorithms using Reinforcement Learning
- Integration of monitoring alerts and automated scaling policies
- Secure secret management using Vault-based solutions
- Real-time log aggregation using Fluentd or Logstash
- Addition of model retraining pipelines and MLOps lifecycle automation

Future versions may also incorporate multilingual symptom understanding and conversational healthcare chatbot capabilities using large language models.

The project establishes a strong foundation for building production-grade AI-enabled healthcare recommendation systems using modern DevOps and MLOps practices.